

Exercise 01:

Create a file named "data.txt" in the same directory as your program. Populate this file with some text content (e.g., "Hello, Memory Mapping!").

Write a C program that performs the following tasks:

- Open the "data.txt" file for reading and writing.
- Uses the mmap system call to map the contents of "data.txt" into memory.
- Displays the content of the mapped memory region on the console.
- Modifies the content of the mapped memory region to append " - Updated!" to the existing text.
- Displays the updated content of the mapped memory region on the console.
- Unmaps the memory region and closes the file.

Exercise 02:

Write C/C++ code for a program that takes as command line argument a file name. Your program will make a memory map of the file and replace all integers with spaces. Make two threads for this purpose. The first thread will replace integers with spaces from the first half of the map and the second thread will replace integers with spaces from the second half of the map. Assume that there are 100 bytes in the file. (Hint: Create a single map. Pass the map pointer to the first thread as a parameter. For the second thread, add 50 to the map pointer and pass it to the second thread as a parameter, i.e., map+50.)

Sample Data for File:

v1gU6OTgN7DMifG7zmQWp04ZEyGmRifq1uFsS9RzZWcCQL7jBMNKUQVEAIsKZia40M3TqJeGEMekk
SagfUc7mU3PbQ1zsiJm23Hq

Exercise 03:

Write three independent C programs (process_A.c, process_B.c, and process_C.c) that utilize **Named Semaphores** to enforce a strict execution sequence. Regardless of the order in which these programs are manually started from the terminal, your implementation must guarantee that the output always follows the order: **Process A → Process B → Process C**. **Process A** should perform a simulated task (e.g., printing its ID and sleeping for 2 seconds) and then signal the first semaphore to "wake up" the next process in line. **Process B** must wait for the signal from Process A before executing its task, then signal the second semaphore to permit Process C to proceed. **Process C** must wait for the signal from Process B before executing its final task.